

CONECTAR A UNA API

PASO 1 : permiso a nuestra app en manifest

```
<uses-permission android:name="android.permission.INTERNET"/>
```

esto va justo encima de <application

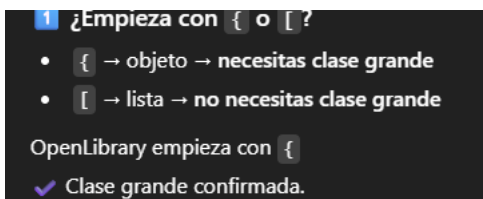
PASO2: añadir la librerías necesarias en el gradle del módulo de la app.

Las librerías son:

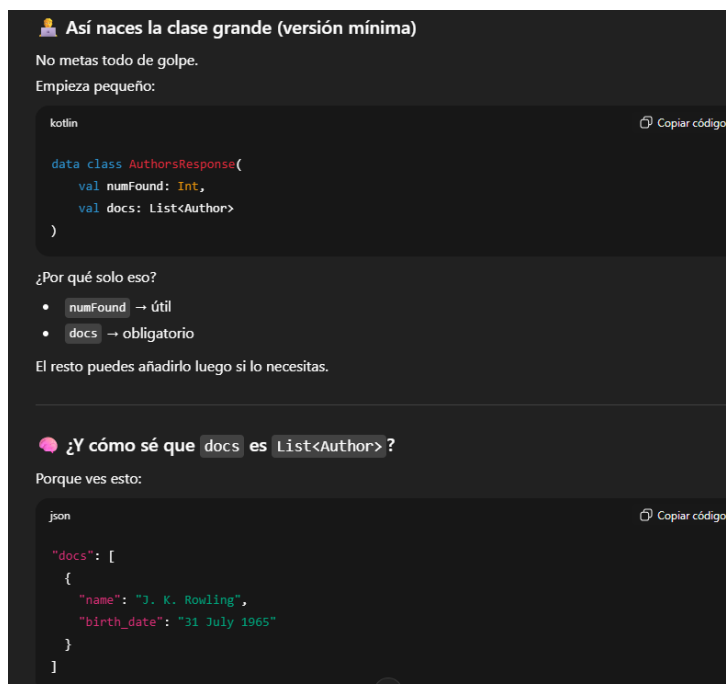
```
implementation ("com.squareup.retrofit2:retrofit:2.11.0")
```

```
implementation ("com.squareup.retrofit2:converter-gson:2.11.0")
```

PASO 3: Mirar el json



```
{
  numFound: 1,
  start: 0,
  numFoundExact: true,
  docs: [
    {
      key: "OL23919A",
      text: [...],
      type: "author",
      name: "J. K. Rowling",
      alternate_names: [...],
      birth_date: "31 July 1965",
      top_work: "Harry Potter and the Philosopher's Stone",
      work_count: 162,
      top_subjects: [...],
      _version_: 1702166143068799000
    }
  ],
}
```



PASO 4: Crear la interfaz ApiService y copiar esto.

```
package com.dam.retrofit2

import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Url

interface ApiService {
    @GET
    suspend fun getAutores(
        @Url url:String): Response<AuthorsResponse>
}
```

PASO 5: crear la funcion getRetrofit()

```
private fun getRetrofit(): Retrofit {
    return Retrofit.Builder()
        .baseUrl("https://openlibrary.org/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()
}
```

muy importante la url debe acabar en / o crasea

PASO 6 : Crear el servicio

fuera de onCreate

private lateinit var apiService: ApiService

dentro de onCreate

apiService = getRetrofit().create(ApiService::class.java)

si hay recycler pega esto fuera de onCreate

```
private fun busquedaAutores(query: String) {
    lifecycleScope.launch(Dispatchers.IO) {

        val call = getRetrofit().create(ApiService::class.java)
            .getAutores("/search/authors.json?q=$query")
        val autoresAPI = call.body()
        if (call.isSuccessful) {
            val autores = autoresAPI?.autores ?: emptyList()
            withContext(Dispatchers.Main) {
                adaptadorRecyclerView.changelist(autores)
            }
        } else {
            withContext(Dispatchers.Main) {
                Toast.makeText(applicationContext, "error API", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

```
}
}
```

si no hay recylcer pega esto fuera de oncreate

```
private fun busquedaAutores(query: String) {
    lifecycleScope.launch(Dispatchers.IO) {

        val response = apiService.getAutores("/search/authors.json?q=$query")

        if (response.isSuccessful) {
            val autores = response.body()?.docs?: emptyList()

            withContext(Dispatchers.Main) {
                Toast.makeText(
                    applicationContext,
                    "Autores encontrados: ${autores.size}",
                    Toast.LENGTH_SHORT
                ).show()
            }

        } else {
            withContext(Dispatchers.Main) {
                Toast.makeText(
                    applicationContext,
                    "Error API",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }
    }
}
```

PARA CREAR UN RECYCLER CON DATOS DE API ESE ES EL ADAPTER

```
class AutoresViewHolder (view: View): RecyclerView.ViewHolder(view) {
    var nombre: TextView
    init{
        nombre=view.findViewById(R.id.tvAutor)
    }
}
```

```
class AdaptadorAutores : RecyclerView.Adapter<AutoresViewHolder> {
    private var aut: List<Autores>
    fun changelist(autores: List<Autores>) {
        aut = autores
        notifyDataSetChanged()
    }

    constructor() {
        aut = ArrayList()
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AutoresViewHolder {
        val inflater = LayoutInflater.from(parent.context)
        return AutoresViewHolder(inflater.inflate(R.layout.fila_autores, parent, false))
    }
}
```

```
}  
  
override fun getItemCount(): Int = aut.size  
override fun onBindViewHolder(holder: AutoresViewHolder, position: Int) {  
    val item = aut[position]  
    holder.nombre.text = item.name ?: "Autor sin nombre"  
  
}  
}
```

el adaptador en el main tambien cambia

```
binding.recycler.layoutManager = LinearLayoutManager(  
    applicationContext,  
    LinearLayoutManager.VERTICAL, false  
)  
binding.recycler.adapter = DatosAdapter()  
}
```

BBDD

PASO1: Dependencias (Gradle del módulo app)

En build.gradle.kts (Module: app) mete:

```
implementation("androidx.room:room-runtime:2.8.3")
```

```
ksp("androidx.room:room-compiler:2.8.3")
```

```
implementation("androidx.room:room-ktx:2.8.3")
```

EN GRADLE DEL PROYECTO DENTRO DE PLUGINS

```
id("com.google.devtools.ksp") version "2.0.21-1.0.27" apply false
```

En build.gradle.kts (Module: app)VUELVE A METER EN PLUGINS

```
id("com.google.devtools.ksp")
```

PASO 2: Crear la tabla (Entity)

crear una clase que representa a una tabla de la base de datos, es como hacer un POJO.

Ejemplo:

```
@Entity(tableName = "favoritos")
```

```
data class AutorEntity(  
    @PrimaryKey
```

```
    @ColumnInfo(name = "autor_key")
```

```
    val key: String,
```

```
    val name: String
```

```
)
```

*los objetos son planos, o sea declaras el tipo de dato que va a tener la tabla.

PASO 3: crear la interface (Crear el DAO) (consultas SQL)

El DAO es la interfaz que define lo que puedes hacer con la tabla: insertar, leer, borrar.

```
@Dao
```

```
interface DatosDao {
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insert(fav: AutorEntity)
```

```
    @Query("SELECT * FROM favoritos")
```

```

suspend fun getAll(): List<AutorEntity>

@Query("DELETE FROM favoritos WHERE autor_key = :key")
suspend fun deleteByKey(key: String)
}

```

PASO 4: CREAR LA BASE DE DATOS (crear la clase que equivale a la base de datos)

codigo del proyecto de VmappWeb (amiguete)

```

@Database(entities = [PostFavoritos::class], version = 1, exportSchema = false)
abstract class PostBBDD: RoomDatabase(){

    abstract fun favDAO(): PostDAO
    companion object DatabaseBuilder{
        private var INSTANCE : PostBBDD ? = null
        fun getInstance (context: Context): PostBBDD {
            if (INSTANCE == null) synchronized(PostFavoritos::class) {
                INSTANCE = buildRoomDB(context)
            }
            return INSTANCE!!
        }
        private fun buildRoomDB (contexto : Context) =
            Room.databaseBuilder (
                contexto.applicationContext, PostBBDD::class.java, "posts.db"
            ).build ()
    }
}

```

paso 5: Iniciar la base de datos donde la vayas a usar (en la practica de amiguete en la activity favoritos)

```

lifecycleScope.launch(Dispatchers.IO) {
    val database = PostBBDD.DatabaseBuilder.getInstance(applicationContext)
    val favs = database.favDAO().selectAll()
    withContext(Dispatchers.Main) {
        adapter.setPosts(favs)
    }
}

```

esto es la courutine con la bbdd dentro.

PASO 6: Guardar favoritos (INSERT) al pulsar el botón

Esto pasa en tu adapter principal (DatosAdapter). Tú ya insertas así:

- Creas `AutorEntity(key, name)`
- Insertas con `coroutine IO`

`DatosAdapter`

```
CoroutineScope(Dispatchers.IO).launch {  
    db.datosDao().insert(favorito)  
}
```

✅ Esto es “Guardar en la BBDD”.

```
holder.toolbar.setOnMenuItemClickListener {  
    when (it.itemId) {  
        R.id.menu_cardview -> {  
  
            val favorito = AutorEntity(  
                key = item.key,  
                name = item.name ?: "Sin nombre")  
            CoroutineScope(Dispatchers.IO).launch {  
                db.datosDao().insert(favorito)  
            }  
  
            true  
        }  
        else -> false  
    }  
}
```

esto esta en el menu de la toolbar del cardview , al pular en fav
pues pasa eso

PASO 8: Borrar (DELETE) desde el Recycler de Activity2

Esto lo haces en FavoritosAdapter

- click en toolbar del item
- deleteByKey(autor.key) en IO
- removeAt(position) + notifyItemRemoved(position) para quitar el card de la UI

```
holder.getToolbar().setOnMenuItemClickListener {  
    when(it.itemId){  
        R.id.action_borrar ->{  
            listener.postDelete(arrayPosts[position])  
            true  
        }  
        else -> {true}  
    }  
}
```